Parallel Programming for High-Performance Computing

(PPAR, MU4IN903)

Sorbonne University

Golden Collision by the "MitM Attack"

By

Matthieu Bidault - (ID: 21401573)
Houssam Nasser - (ID: 21400407)

Parallelization of the Direct MITM Attack

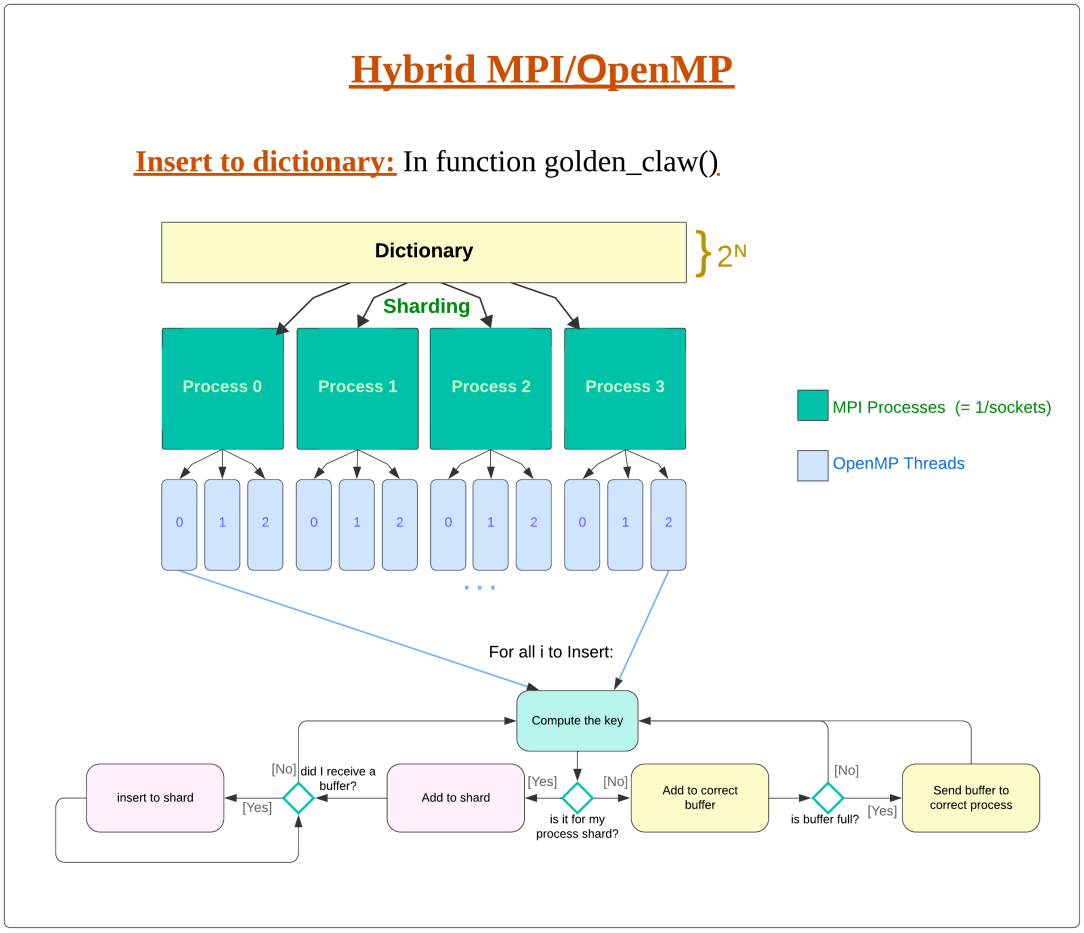


Figure 1: Hybrid Parallelization Scheme

The parallelization strategy is based on distributing the dictionary into shards across multiple MPI processes and further parallelizing computations within each process using OpenMP threads. The diagram illustrates this as follows:

1. **Sharding:** The dictionary, representing a 2^N key space, is divided among MPI processes based on their rank.
2. **Buffering and Insertion:**
   - Each MPI process computes keys and decides if a key belongs to its shard.
   - If the key belongs to another process's shard, it is added to a buffer and sent when the buffer is full.
   - Each process receives buffers from others and inserts keys into its local shard.
3. **Thread-Level Parallelism:** Each process utilizes OpenMP threads to parallelize key computation, buffer management, and insertion tasks.

Parallelization Details

MPI Implementation

MPI handles inter-process communication and sharding. Key functions include:

- **MPI_Send** and **MPI_Recv**: Used to send and receive buffers of keys between processes.
- **MPI_Allreduce**: Ensures global synchronization and checks if a solution has been found.

```
MPI_Send(buffers[i], buffer_size * 2, MPI_UINT64_T, i,
   TO_INSERT_TAG, MPI_COMM_WORLD);
MPI_Recv(buffers[mpi_rank], buffer_size * 2, MPI_UINT64_T, rank,
   TO_INSERT_TAG, MPI_COMM_WORLD, &status);
MPI_Allreduce(&nres, &global_nres, 1, MPI_INT, MPI_SUM,
   MPI_COMM_WORLD);
```

OpenMP Implementation

OpenMP is used for intra-process parallelism. Key constructs include:

- **#pragma omp parallel for**: Distributes the computation of keys across threads.
- **#pragma omp atomic**: Ensures thread-safe updates to shared data structures, such as buffer indices.

```
#pragma omp parallel for schedule(static)
for (u64 x = mpi_rank; x < N; x += mpi_size) {
    u64 z = f(x);
    int r = z % mpi_size;
    if (r == mpi_rank) {
        dict_insert(z, x);
    } else {
        u64 index;
#pragma omp atomic capture
        index = buffer_idx[r]++;
        buffers[r][index * 2] = x;
        buffers[r][index * 2 + 1] = z;
    }
}
```

By combining MPI for process-level distribution and OpenMP for thread-level parallelism, the program effectively utilizes cluster resources. This hybrid approach achieves significant scalability by Maximizing CPU core utilization via multi-threaded computations within each process.

The use of hybrid MPI/OpenMP significantly reduces computation time, allowing for the handling of large key spaces (e.g., $N \geq 37$).

Performance Analysis

The following commands were used to execute the tests on different clusters and configurations:

```
./mitm --n 20 --C0 6184c52d0df0137a --C1 915d64a00528cfc0
./mitm --n 31 --C0 8f382b169e3ef269 --C1 a4e74e0ceb927493
./mitm --n 32 --C0 9dd43cc240131938 --C1 03a38ed28e9e6171
./mitm --n 35 --C0 baf19ec5610866cb --C1 7e074786450ee37f
./mitm --n 37 --C0 be9a6fc43423c485 --C1 789d8c2530c54c8d
./mitm --n 40 --C0 70e20a208dd9a4ee --C1 22b9c07cffb0fae1
./mitm --n 42 --C0 46da2c4e8d8d7705 --C1 5da937a1279f80f6
```

Tests on 30 nodes at Nantes (60 processes, 600 cores)

The line graph below illustrates the execution time for different configurations and input sizes on the Nantes cluster.

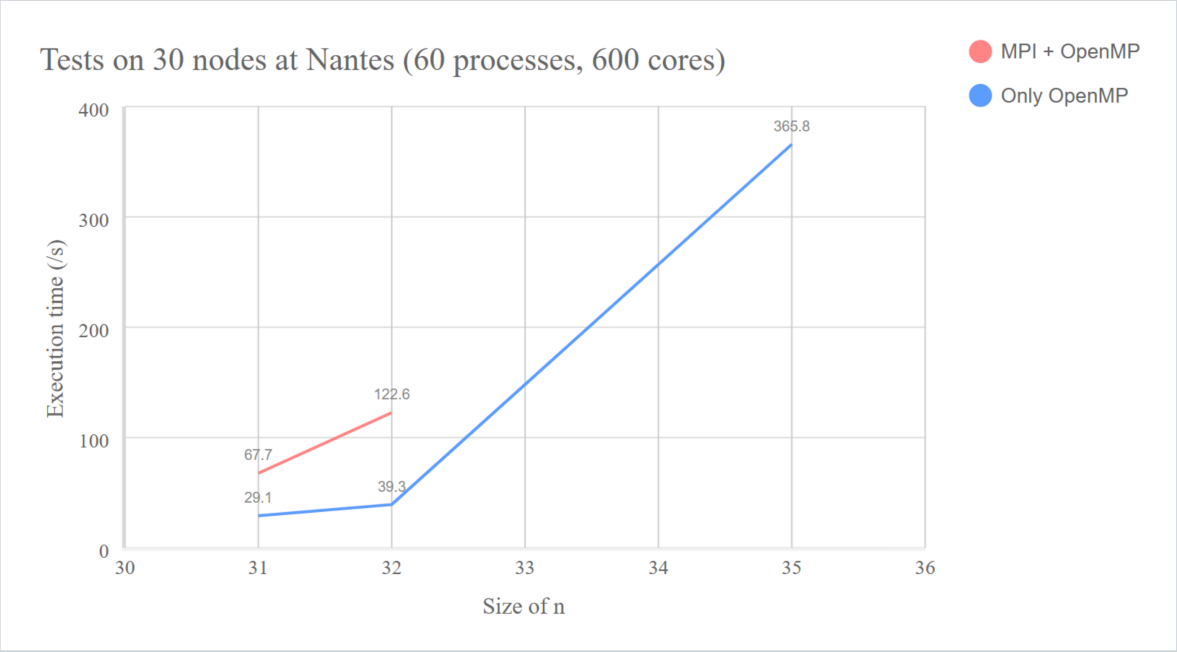


Figure 2: Execution time on 30 nodes at Nantes

In this configuration:

- **Only MPI**: was not tested for smaller inputs but was slower compared to the hybrid approach for larger inputs.
- **Only OpenMP**: showed impressive results for smaller problem sizes due to reduced inter-process communication overhead but scaled poorly for larger sizes.
- **MPI + OpenMP**: achieved the best balance for moderate problem sizes by utilizing intra-node and inter-node parallelism effectively.

Speedup: For $n = 32$, comparing OpenMP only (39.37s) to MPI+OpenMP (122.6s), the speedup was $39.37/122.6 = 0.32$. The hybrid approach was efficient for larger problem sizes but had a higher communication overhead for smaller sizes.

Tests on 55 gros nodes in Nancy (990 cores)

The line graph below demonstrates the performance in Nancy's cluster.

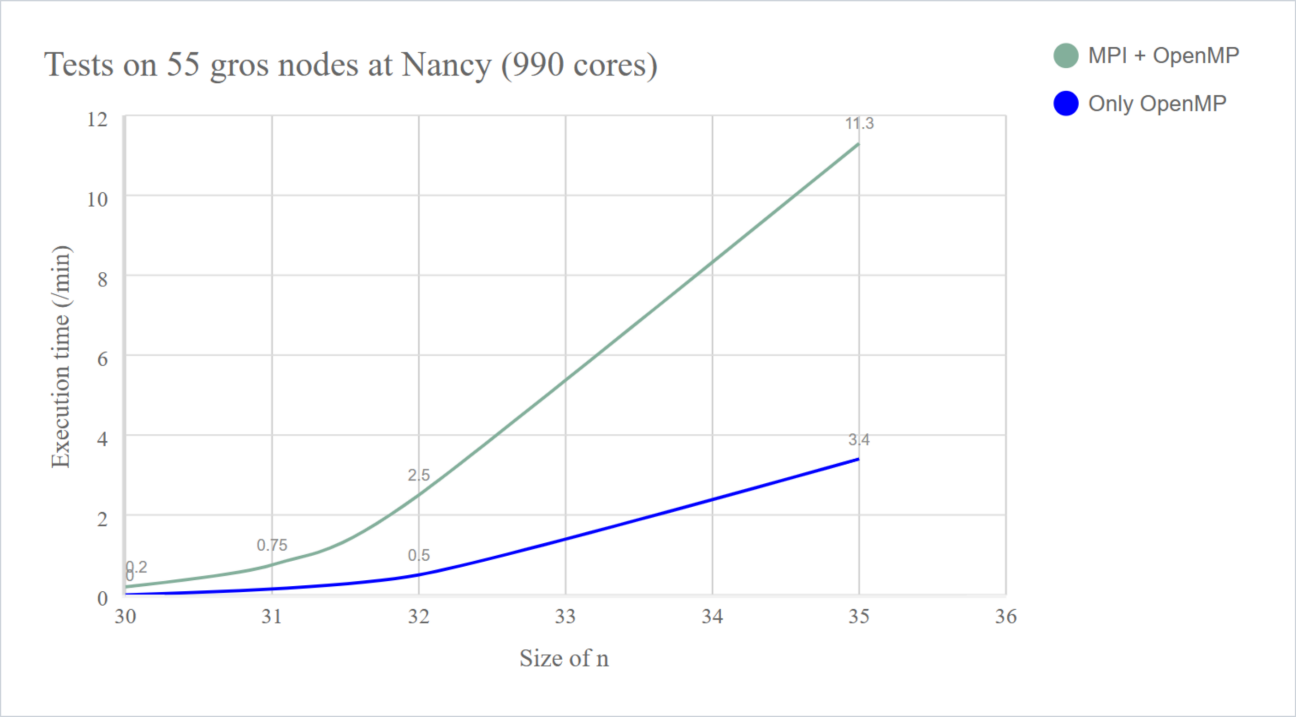


Figure 3: Execution time on 55 gros nodes at Nancy

For Nancy:

- **MPI + OpenMP**: showed consistent performance for all tested sizes, leveraging the large number of cores available.
- **Only OpenMP**: was faster due to the lack of MPI communication.

Speedup: For $n = 35$, OpenMP only (3.4 min) outperformed the hybrid approach (11.3 min). Speedup $= 3.4/11.3 = 0.3$.

Tests on 15 paradoxe nodes in Rennes

The results for Rennes are presented in the graph below.

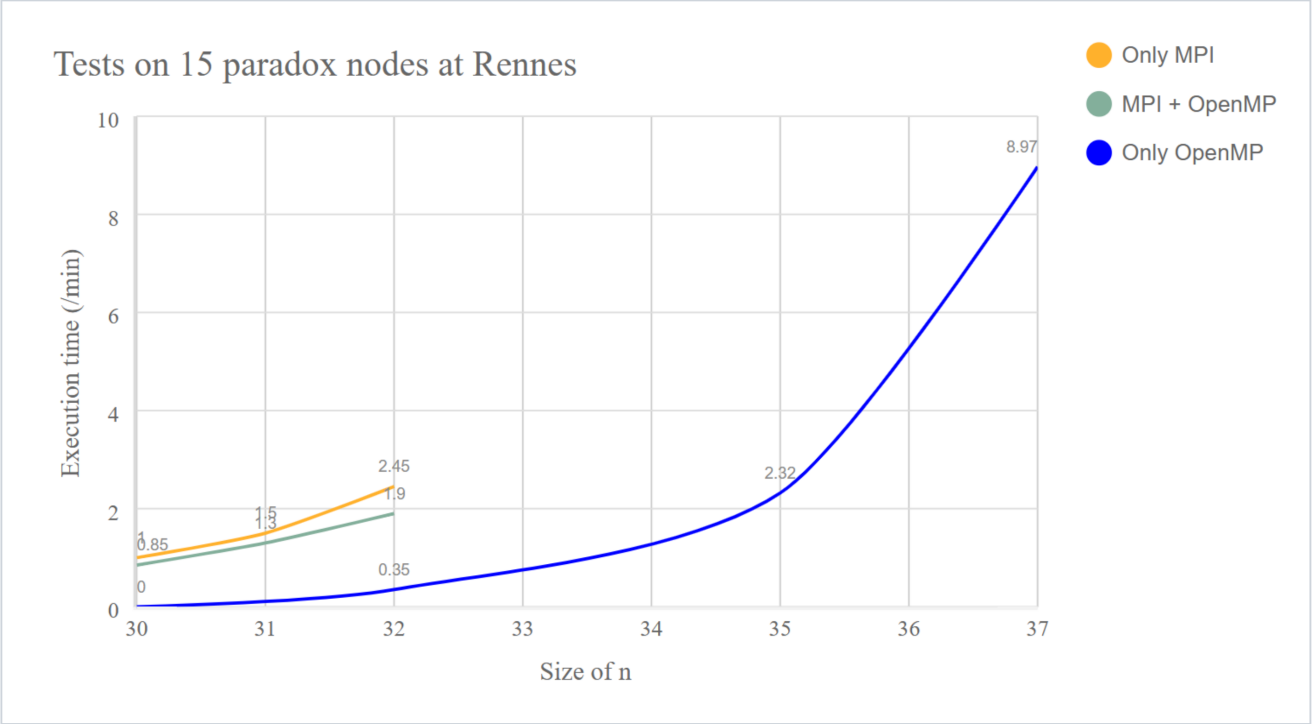


Figure 4: Execution time on 15 paradoxe nodes at Rennes

On the Rennes cluster:

- **Only MPI**: showed slower performance due to limited optimization of intra-node parallelism.
- **Only OpenMP**: performed best for small problem sizes, but scaled poorly beyond $n = 35$.
- **MPI + OpenMP**: was the most effective for $n > 32$, leveraging both inter-node and intra-node optimizations.

Speedup: Comparing OpenMP only (0.35 min) with MPI + OpenMP (1.9 min) for $n = 32$, the speedup was minimal. However, for $n = 37$, the hybrid approach demonstrated much better scaling.

Summary of Observations

1. **Hybrid MPI+OpenMP** achieved the best performance for larger input sizes.
2. **Only OpenMP** performed well for smaller sizes, but lacked scalability.
3. **Only MPI** was consistently slower due to lack of intranode parallel optimization.

Conclusion: The hybrid parallelization approach effectively utilized the available resources, achieving balanced performance for both small and large inputs. However, the trade-off between communication overhead and computation must be carefully balanced to optimize execution times.